Numerical Integration and Error Analysis

Scientific Computing Project 7

4 November 2026

Abstract

This project implements and evaluates the composite trapezoidal rule and the composite Simpson's one-third rule. The implementation includes input validation, absolute-error calculations, observed convergence-order estimates, automated experiments, visualizations, and unit tests. Results on smooth functions agree with classical theory: the trapezoidal rule converges as $O(h^2)$ and Simpson's rule as $O(h^4)$. Additional experiments show how resolution, polynomial exactness, oscillation, sharp peaks, endpoint singularities, and floating-point roundoff affect these nominal orders.

Contents

1 Introduction and objectives **1**

2 Mathematical background **1**
2.1 Composite trapezoidal rule 2
2.2 Composite Simpson's one-third rule 2
2.3 Error and observed order 2

3 Software design and implementation **2**

4 Experimental setup **3**

5 Results **3**
5.1 Smooth functions 3
5.2 Polynomial exactness 4
5.3 Rapid oscillation 4
5.4 Sharp peak 4
5.5 Endpoint singularity 4

6 Verification and testing **4**

7 Discussion and limitations **4**

8 Conclusion **5**

1 Introduction and objectives

Definite integrals frequently cannot be evaluated in closed form, or their integrands may only be available through samples. Numerical quadrature replaces the integral by a weighted sum of

function values. The aim of this project is to implement two standard composite quadrature methods and study their accuracy empirically.

The project objectives are:

1. implement the composite trapezoidal rule;
2. implement the composite Simpson's one-third rule;
3. measure absolute errors against known exact integrals;
4. estimate convergence orders as the mesh is refined;
5. compare behavior on smooth, polynomial, oscillatory, sharply peaked, and nonsmooth functions; and
6. validate the implementation with automated tests and input checks.

2 Mathematical background

Let f be integrated over $[a, b]$ using n equal subintervals. The mesh width is

$$h = \frac{b-a}{n}, \qquad x_i = a + ih, \quad i = 0, \ldots, n.$$

2.1 Composite trapezoidal rule

On each subinterval, f is approximated by the line joining its endpoint values. Summing the local trapezoids gives

$$T_n = h \left[\frac{f(x_0) + f(x_n)}{2} + \sum_{i=1}^{n-1} f(x_i) \right].$$

For sufficiently smooth f, the global error is $O(h^2)$; more specifically, it is proportional to $h^2 f''$ in the standard error estimate. The method is exact for all polynomials of degree at most one.

2.2 Composite Simpson's one-third rule

Simpson's rule applies a quadratic interpolant over each pair of subintervals. Therefore n must be positive and even:

$$S_n = \frac{h}{3} \left[f(x_0) + f(x_n) + 4 \sum_{\substack{i=1 \\ i \text{ odd}}}^{n-1} f(x_i) + 2 \sum_{\substack{i=2 \\ i \text{ even}}}^{n-2} f(x_i) \right].$$

For sufficiently smooth f, its global error is $O(h^4)$ and depends on the fourth derivative. Simpson's rule is exact for polynomials of degree at most three.

2.3 Error and observed order

For an exact integral I and numerical approximation Q_n, the project uses

$$E_n = |Q_n - I|.$$

Given two mesh widths and errors, the observed order is

$$p = \frac{\log(E_1/E_2)}{\log(h_1/h_2)}.$$

When the error is at or below 10^{-14}, the code does not report an order because roundoff can dominate the discretization error.

3 Software design and implementation

The source is organized into independent modules:

- `src/trapezoidal.py`: composite trapezoidal quadrature;
- `src/simpson.py`: composite Simpson quadrature using NumPy arrays;
- `src/error_analysis.py`: validation, error computation, convergence studies, and table formatting;
- `src/experiments.py`: reproducible experiments and plots; and
- `tests/test_error_analysis.py`: automated tests.

Both quadrature functions verify that the integrand is callable, endpoints are real and finite, and n is a positive integer. Simpson's implementation additionally rejects odd n. The analysis utilities reject invalid endpoints, duplicate or nonpositive subdivision counts, nonfinite exact values, negative errors, and invalid step sizes. Reversed integration limits are supported; the step size used for reporting is positive while the quadrature value retains the correct sign.

The experiment driver uses

$$n \in \{2, 4, 8, 16, 32, 64, 128, 256, 512, 1024\},$$

which is suitable for both methods because every value is even. It writes complete numerical tables to `results/tables.txt` and produces log–log error plots and a summary comparison plot.

4 Experimental setup

The following exact integrals were used:

Function	Interval	Exact integral
e^x	$[0, 1]$	$e - 1$
$\sin x$	$[0, \pi]$	2
x^k	$[0, 1]$	$1/(k + 1)$
$\sin(20x)$	$[0, 1]$	$(1 - \cos 20)/20$
$1/(1 + 25x^2)$	$[-1, 1]$	$2 \arctan(5)/5$
$\sqrt{x}$	$[0, 1]$	$2/3$

The experiments were run using the project's generated results. The polynomial study used $n = 4$ and degrees $k = 0, \ldots, 5$. A result was labeled exact when its absolute error was at most 10^{-14}.

5 Results

5.1 Smooth functions

For e^x on $[0, 1]$, at $n = 16$ the trapezoidal error is 5.593×10^{-4} and the Simpson error is 1.456×10^{-7}. Simpson's rule is therefore approximately 3.8×10^3 times more accurate at that mesh size. The observed orders approach 2 and 4, respectively.

For $\sin x$ on $[0, \pi]$, the final errors at $n = 1024$ are 1.569×10^{-6} for the trapezoidal rule and 9.841×10^{-13} for Simpson's rule. The observed orders remain close to the theoretical values throughout refinement.

Table 1: Representative smooth-function results at $n = 16$.

Function	Trapezoidal error	Simpson error	Error ratio
e^x	5.593×10^{-4}	1.456×10^{-7}	3.84×10^3
$\sin x$	6.430×10^{-3}	1.659×10^{-5}	3.88×10^2

5.2 Polynomial exactness

The polynomial study confirms the interpolation properties. Both methods are exact for degrees zero and one. Simpson's rule remains exact for degrees two and three, while the trapezoidal rule is no longer exact. At degree four, the errors are 2.070×10^{-2} and 5.208×10^{-4} for the trapezoidal and Simpson rules, respectively.

5.3 Rapid oscillation

For $\sin(20x)$, the period is approximately 0.314, while the initial mesh widths are 0.5 and 0.25. These meshes do not resolve the oscillation. Consequently, the trapezoidal error increases from 7.337×10^{-2} at $n = 2$ to 1.286×10^{-1} at $n = 4$, producing an apparent order of -0.8101. Simpson's corresponding early order is 0.7073. Once the mesh becomes sufficiently fine, the orders settle near two and four.

5.4 Sharp peak

For $f(x) = 1/(1 + 25x^2)$, the peak near zero is initially under-resolved. At $n = 8$ and $n = 16$, Simpson's errors are 2.588×10^{-2} and 2.696×10^{-3}, whereas trapezoidal errors are 7.538×10^{-3} and 1.380×10^{-4}. Simpson is temporarily worse. During the transition from unresolved to resolved, the reported Simpson orders 7.2139 and 10.9629 are not genuine asymptotic rates; after $n \geq 128$, the order stabilizes near four.

5.5 Endpoint singularity

For $\sqrt{x}$, the derivative $f'(x) = 1/(2\sqrt{x})$ is unbounded at zero. The smoothness assumptions behind the nominal orders fail. At $n = 1024$, the observed orders are 1.4962 for the trapezoidal rule and 1.5000 for Simpson's rule. Simpson still has the smaller error, 2.478×10^{-6} versus 6.304×10^{-6},

but it does not achieve fourth-order convergence. A standard remedy is $x = t^2$, which transforms the integral into $\int_0^1 2t^2\,dt$.

6 Verification and testing

The test suite contains 50 passing tests. It checks basic error calculations, NumPy scalar compatibility, rejection of nonreal and nonfinite values, known-order recovery, symmetry of order estimation, zero-error handling, interval reversal, invalid subdivision counts, propagation of method errors, polynomial exactness, convergence of both rules, and roundoff-floor behavior. The successful test run provides evidence that both the numerical methods and the analysis utilities meet their intended contracts.

7 Discussion and limitations

The experiments demonstrate that a theoretical convergence order is an asymptotic statement, not a guarantee at every mesh size. A method may appear worse before a function is resolved, especially for oscillatory or localized features. The error constants also matter: fourth-order convergence does not automatically imply a smaller error on a coarse grid. In addition, finite precision eventually limits meaningful order estimates.

The current implementation assumes equally spaced nodes and scalar function evaluation. It does not adaptively refine the mesh, estimate unknown exact integrals, or treat singularities automatically. Simpson's implementation uses NumPy, while the trapezoidal implementation evaluates points in a Python loop; vectorization and adaptive quadrature would be natural extensions.

8 Conclusion

The project successfully implements and tests two fundamental composite quadrature methods. For smooth functions, the observed orders agree with theory: $O(h^2)$ for the trapezoidal rule and $O(h^4)$ for Simpson's rule. Simpson's rule is generally substantially more accurate at comparable resolution, but the challenging examples show why smoothness and adequate resolution are essential. Polynomial exactness, pre-asymptotic behavior, endpoint singularities, and roundoff all influence the observed results. Overall, the code provides a clear and reproducible framework for numerical integration error analysis.

Reproducibility

From the project root, install the dependencies and run:

```
python src/experiments.py
pytest -q
```

The first command regenerates the tables and figures in `results/`; the second runs the automated verification suite.